Parallel CSV Data Processing Pipeline

M Ahmed Khurram (24i0005)
Afaan Sohail (24i0081)
Abdul Hannan (24i2056)

Problem Statement:
This project basically aims to create a pipeline based on OS concepts where the user just runs a single command and the entirety of data reading, processing and taking out meaningful information from it is done by a program that handles similar yet independent tasks concurrently saving time.

System Design and Implementation Details:
A main process creates three IPC structures. A named pipe for data transfer between ingestor and processor, a shared memory to hold final results and a names semaphore to synchronize these processes.
It then used fork() system call to create three child processes and then uses exec to replace its memory with relevant codes (ingestor, processor and reporter). It also uses dup2 to redirect the output from child processes form terminal to log files.
After this the dispatcher goes into wait until the children have completed their tasks or have been terminated.

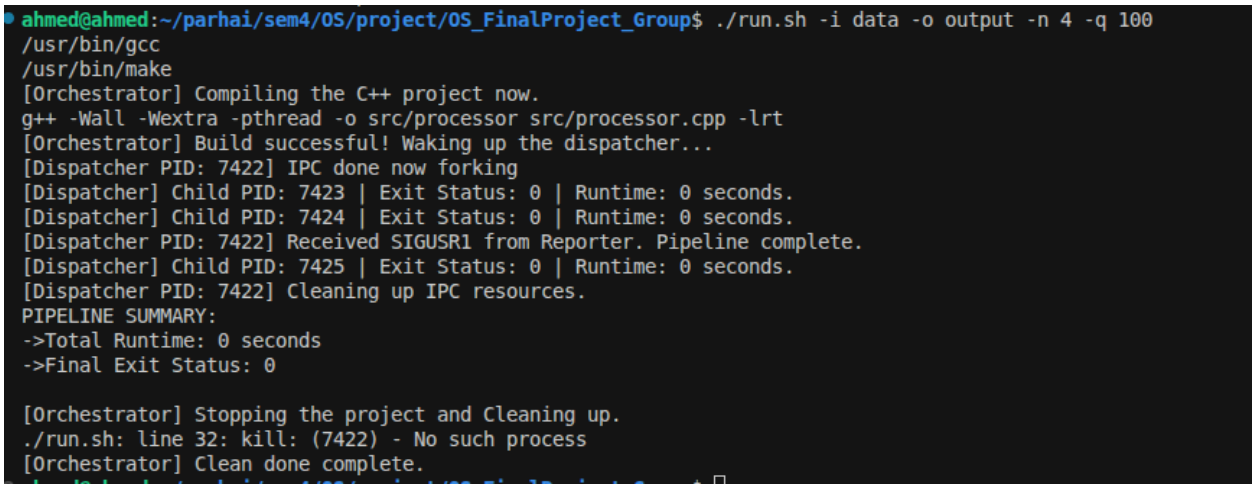
The ingestor reads csv file into a chunk struct and then writes it to the named pipe after giving the chunk an ID. The ingestor after writing all chunks to the names fifo sends a chunk with and ID of -1 that tell the processor that data stream is over. After this the ingestor exits.

While the ingestor write to the fifo, processor on the other end keeps on reading it and pushes them into a bounded task queue. The processor creates 4 worker threads that act as consumers that constantly pull chunks out the task queue and processes it. Processing includes parsing the csv, calculating the moving avg, checking for anomalies and updating the aggregation table. They make use of mutex locks to ensure that no multiple threads try to update the same data simultaneously.
After the threads have done their work the aggregation table is converted to a flat array of finalrecord struct by the processor. It writes this array to the shared memory initially created by the dispatcher. It then calls sem_post on the named semaphore to let the reporter know it has to take over now.

The reporter now opens the share memory and read the contents until it reaches null character. It loops through entire records to write them to the report.csv. It uses dup2 to redirect output from terminal to a report.txt file and creates a table there for final stats. It lastly sends a signal to the dispatcher letting it know the tasks have been completed and then it exits.

The signal from the reporter unpauses the dispatcher which collects the exit statuses of each child process and cleans up the work space by deleting fifo, shared memory and the semaphore. It then exits gracefully.

Sample Output:



Work Division:
All three of us had sat down and finalized the entire architecture of how the pipeline would be designed and then delegated tasks accordingly:

- Abdul Hannan coded dispatcher.cpp and ingestor.cpp along with makefile and run file
- Afaan Sohail coded processor.cpp and handled ipc between threads
- Ahmed Khurram coded reporter.cpp, readme and wrote the report for the project

Regular meetings were held to ensure we were on track and to discuss any ideas that later came to mind.

Challenges Faced:
Initial challenge was to design the pipeline in accordance with OS concepts we had to use as there we many different ways this could be done. Moving forward a problem we later faced was handling different types of cli arguments that could be passed for custom execution of the pipeline. A major problem was the issue of accessing the global hash table (aggregation table) each time a thread had to store data. Initially we were applying mutex locks on the global hash table every time data was aggregated there however this was time consuming. To solve this we created local hash tables for each thread which are only aggregated to the global hash table once the thread has completed it part. For the reporter.cpp entirety of pipeline and its structure had to be understood to develop a reporter that was efficient and followed the same structures.